



CESAR SCHOOL
GRADUAÇÃO EM CIÊNCIA COMPUTAÇÃO

Tópicos Contemporâneos 4: Fundamentos de Big Data
Turma 8A - 2026.1

Relatório Técnico - AV1

Equipe 10:

Ana Beatriz Alves (abxa@cesar.school)
Caio Barreto (cba2@cesar.school)
Diego Arruda (dca2@cesar.school)
Henrique Roma (hrm@cesar.school)
João Henrique Lafetá (jhml@cesar.school)
Rodrigo Dubeux (rdmo@cesar.school)
Victor Hora (vht@cesar.school)

SUMÁRIO

1. Contexto	1
2. Checklist AV1	2
3. Diagrama de Arquitetura	2
4. Tecnologias utilizadas e Planejamento	3
5. Arquitetura Implementada	5
6. Equipe e Divisão de Tarefas	7
7. Funções desempenhadas	8
8. Links úteis	9

1. Contexto

Operando como um pilar essencial do transporte público em uma metrópole de alta densidade, o Citi Bike NY possui um sistema que gere um volume massivo de dados estruturados e semiestruturados, proveniente de milhões de viagens e integrações externas. Porém, a gestão dessa frota enfrenta desafios constantes de equilíbrio de inventário em tempo real, evidenciada pela lacuna na capacidade de antecipação logística, representado pela ruptura de estoque (estações que ficam sem bicicletas disponíveis em horários de pico) e superlotação (estações em polos comerciais que atingem a capacidade máxima, impedindo que usuários finalizem suas viagens).

Além disso, os custos reativos associados a essas variações, como, por exemplo, o rebalanceamento da frota (movimentação de bicicletas via caminhões) são frequentemente empregados de forma reativa, gerando gastos operacionais elevados e ineficientes.

Dessa forma, este projeto foca no desenvolvimento de uma solução de Big Data voltada para a mobilidade urbana, especificamente no sistema de compartilhamento de bicicletas de Nova York, o Citi Bike. Propomos a implementação de um pipeline de Big Data robusto que integre dados históricos de viagens com variáveis climáticas horárias e calendários de feriados. Utilizando da inteligência aplicada e potencializada pelo Big Data, a solução empregará modelos preditivos para antecipar a demanda de passageiros por rota e horário, visando reduzir custos logísticos de rebalanceamento e maximizar a receita operacional por estação.

Dessa forma, adotando o paradigma da Arquitetura Lambda Híbrida combinada com a Arquitetura Medallion (Bronze, Silver, Gold), tem o objetivo de suportar tanto o processamento massivo de dados históricos (Batch Layer) quanto a emulação de fluxos em tempo real (Speed Layer) para alertas logísticos.

Atualmente, o sistema opera em um ambiente simulado (Local/Virtual Environment), escolhida para evitar custos de nuvem durante o desenvolvimento, utilizando o sistema de arquivos local como Mock Data Lake e a memória da máquina para o processamento distribuído via PySpark. Embora execute localmente, toda a

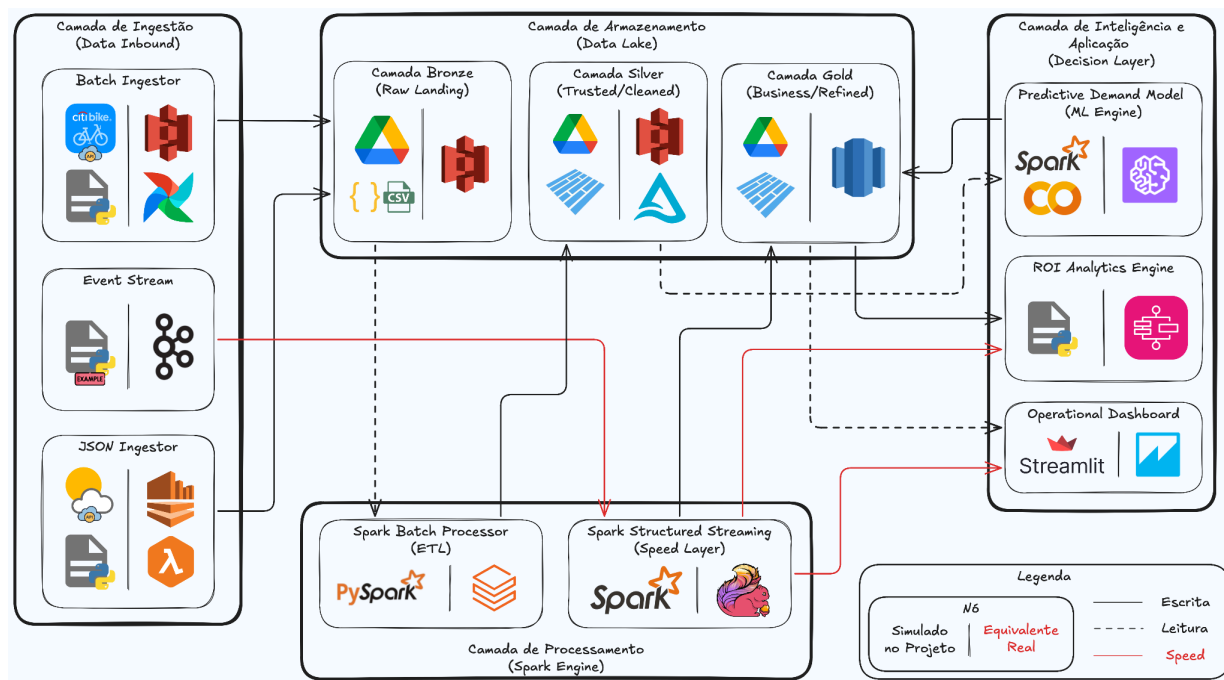
base de código (ex: uso do formato Parquet e transações ACID com Delta Lake) é 100% Cloud-Ready, podendo ser migrada para um cluster real sem alterações no core da engenharia.

2. Checklist AV1

Etapa	Status
Ingestão	✓ Finalizado
Armazenamento	✓ Finalizado
Transformação	✓ Finalizado

3. Diagrama de Arquitetura

O fluxo implementado nos módulos (notebooks localizados na pasta `/dados`) engloba Ingestão, Armazenamento e Transformação, mapeado da seguinte forma:



a. Camada de Ingestão (Data Inbound)

Esta camada é responsável por capturar os dados do mundo exterior e trazê-los para o sistema. Temos o Batch Ingestor, responsável por capturar dados históricos pesados (viagens), o Event Stream, responsável por capturar o fluxo de viagens acontecendo em tempo real, e, por fim, os JSON Ingestors, responsáveis por capturar dados de contexto em formato JSON (Clima, Feriados).

b. Camada de Armazenamento (Data Lake / Medallion)

Local onde os dados descansam em diferentes níveis de qualidade, temos a Camada Bronze, onde o dado chega bruto, do jeito que foi extraído, a Camada Silver, onde o dado já foi limpo, filtrado e padronizado, e, na última etapa, a Camada Gold, onde o dado está agregado e pronto para gerar valor financeiro.

c. Camada de Processamento (Spark Engine)

O "músculo" da operação, responsável por transformar os dados entre as camadas do Data Lake, composto pelo Spark Batch Processor, que lê grandes volumes de uma vez, limpa e salva, e pelo Spark Structured Streaming, que fica "escutando" eventos em tempo real sem parar.

d. Camada de Inteligência e Aplicação (Decision Layer)

A ponta final, que os diretores e a operação da empresa vão enxergar, é composto pelo Modelo Preditivo de Demanda (ML Engine), o ROI Analytics Engine, que traduz a logística em dinheiro (lucro/prejuízo) e o Dashboard Operational.

4. Tecnologias utilizadas e Planejamento

A stack tecnológica foi selecionada para garantir viabilidade de execução local, espelhando os padrões da indústria. Abaixo, detalhamos a tecnologia atual e como ela seria substituída em um cenário corporativo com orçamento:

Etapa do Pipeline	Tecnologia Projeto	Tecnologia Cenário Real	Justificativa da Escolha / Refinamento
Armazenamento	File System Local	Amazon S3 / Amazon Redshift	O disco local simula a latência de um bucket de Object Storage. No cenário real, o S3 oferece escalabilidade infinita e desacoplamento entre armazenamento e processamento; Já o Redshift atuaria como o Data Warehouse para consultas SQL de alta performance na Gold.
Streaming Engine	Script Python (yield JSON)	Kafka	Criação de um simulador para emular a pressão da rede de bikes. No mundo real, o Kafka real garante a persistência (retenção) e a entrega at-least-once dos eventos dos totens, mesmo em caso de falha.
Orquestração	Execução manual/linear	Apache Airflow / AWS Step Functions	Scripts Python executam a esteira na execução simulada. No mundo real, o Airflow e o Step Functions substituem a intervenção humana por DAGs automatizadas que gerenciam dependências, retentativas (retries) e alertas em caso de quebra no pipeline.
Processamento Batch	PySpark Local	Databricks	O Spark local valida a lógica de código. No mundo real, o Databricks em escala fornece auto-scaling de clusters, otimizando custos ao alocar máquinas apenas durante o processamento.
Processamento Stream	Spark Streaming	Apache Flink	O Spark trata os dados em micro-lotes (latência de segundos). Já o Flink processa evento-por-evento, essencial para alertas de rebalanceamento que exigem milissegundos.
Formato de Dados	Parquet e Delta Lake	Delta Lake (Enterprise Databricks)	Preferiu-se por usar Delta Lake open-source localmente para garantir transações ACID. No cenário real, o Delta permite que múltiplos processos leiam e escrevam sem corromper os dados.

Camada de Inteligência (ML)	Spark MLlib / Scikit-Learn	Amazon SageMaker	O SageMaker automatiza o ciclo de vida do modelo (MLOps), permitindo que o modelo preditivo seja atualizado e monitorado automaticamente em produção.
Análise de ROI e Custos	Scripts Python Customizados	AWS Step Functions	Automatiza o cálculo financeiro de cada viagem perdida, transformando logs operacionais em métricas de negócio em tempo real para o departamento financeiro.
Visualização Operacional	Streamlit	Amazon QuickSight	O Streamlit foca em agilidade para o protótipo. Ferramentas como QuickSight oferecem maior segurança de dados e capacidade de escala para centenas de usuários corporativos.

5. Arquitetura Implementada

A arquitetura encontra-se parcialmente implementada por dois motivos principais: primeiro, porque atualmente está sendo executado em um ambiente simulado e não numa infraestrutura de nuvem (Cloud) real; segundo, porque apenas a fundação de Engenharia de Dados foi construída até agora, estando a camada de Inteligência e Negócio reservada para a próxima fase (AV2).

a. O Ambiente Simulado (Infraestrutura Mock)

Para evitar os custos associados a serviços de nuvem (como a AWS ou Azure) durante esta fase inicial, a arquitetura foi desenhada para operar localmente, mas mantendo código com padrão de mercado, como, por exemplo, um Data Lake Simulado, um motor de processamento local e um simulador de streaming.

b. A Arquitetura Medallion Implementada (A Base de Dados)

Até ao momento, a equipe conseguiu implementar na íntegra a arquitetura Medallion, garantindo que os dados fluam pelas três camadas de qualidade:

- Camada Bronze: O Notebook 01 (`/notebooks/01_batch_ingestion.ipynb`) vai buscar gigabytes de dados

históricos de viagens (CSV) e de APIs externas (Clima e Estações em JSON), guardando tudo no formato Parquet de forma imutável.

- Camada Silver: O Notebook 03 (`/notebooks/03_silver_layer.ipynb`) processa a camada Bronze, remove duplicados, calcula distâncias (fórmula de Haversine) e faz o cruzamento com o clima. Aqui, introduziu-se o armazenamento em o Delta Lake, que garante que os dados não se corrompem caso o processo falhe a meio (transações ACID).
- Camada Gold: Os Notebooks 04 (`/notebooks/04_gold_layer.ipynb`) e 05 (`/notebooks/05_pyspark_pipeline.ipynb`) criam a camada de alto valor, onde os dados da Silver são transformados num Modelo Estrela (para relatórios de Business Intelligence) e fatiados em janelas temporais contínuas de 15 e 30 minutos (preparando o terreno para o Machine Learning).

c. Próximos passos

A arquitetura é considerada "parcial" porque a ponta final do pipeline ainda não foi construída, faltando implementar:

- O Consumidor de Streaming (Spark Structured Streaming): O processo que vai ficar ativamente à escuta dos ficheiros JSON gerados pelo vosso simulador.
- Modelagem Preditiva (ML Engine): O algoritmo de Machine Learning que vai consumir as janelas de 15/30 minutos da camada Gold para prever a rutura de stock das estações (Notebook 06).
- Motor de ROI e Dashboard: A interface visual e matemática que vai traduzir os acertos logísticos em métricas de poupança financeira (Notebook 07).

Em suma, os componentes mais centrais de extração, limpeza e armazenamento distribuído de Big Data já estão implementados e funcionais.

6. Equipe e Divisão de Tarefas

A equipe foi organizada seguindo a lógica de Pair Programming, em que cada par de integrantes foca em uma especialidade do pipeline para garantir profundidade técnica. Assim, as frentes de trabalho foram divididas da seguinte forma:

a. Frente de Ingestão e Simulação de Eventos

- Ingestão de Dados e Fontes externas: Responsável por coletar os dados históricos do Citi Bike e ingerir fontes secundárias (clima/feriados) para enriquecer o modelo de demanda
- Streaming e Simulador de Eventos: Responsável pela criação do simulador em Python que envia dados de viagens (origem/destino) em tempo real, simulando a "pressão" logística que o sistema sofre ao longo do dia.

b. Frente de Engenharia de Dados e Arquitetura

- Arquitetura Medallion e Armazenamento: Responsável pela implementação das camadas Bronze, Silver e Gold, garantindo que os dados transformados sejam salvos de forma eficiente (Parquet/Delta) para consumo rápido.
- Pipeline PySpark: Responsável pelo desenvolvimento das transformações críticas utilizando PySpark, focando em agregar a demanda por estação e calcular janelas de tempo de 15 a 30 minutos.

c. Frente de Inteligência Logística e Modelagem

- Modelo Preditivo de Demanda: Responsável pelo desenvolvimento do modelo preditivo que antecipa o esvaziamento ou lotação das estações, com meta de gerar o "alerta de rebalanceamento" antes que a crise logística ocorra, maximizando a ocupação.

d. Frente de Economia Empresarial e Visualização

- Business Case e Dashboard ROI: Responsável por traduzir a acurácia do modelo em termos monetários, e pela criação do dashboard que mostra quanto

a empresa economiza ao reduzir caminhões de rebalanceamento e quanto ganha ao não ter estações vazias em horários de pico. Dessa forma, criará os KPIs de Eficiência Logística e ROI:

- Taxa de Ruptura de Estoque (Stock-out Rate): Percentual de tempo em que uma estação fica com 0 bicicletas disponíveis em horários de pico. O modelo visa reduzir esse KPI para maximizar a conversão de aluguéis.
- Custo de Rebalanceamento por Viagem: Valor estimado gasto para mover bicicletas via caminhões entre estações. O objetivo é diminuir esse custo através da predição, movendo as bikes apenas quando estritamente necessário.
- Receita Potencial Recuperada: Cálculo de quantas viagens deixaram de ser perdidas devido à disponibilidade garantida pelo modelo preditivo.
- Índice de Rotatividade do Ativo: Quantas vezes cada bicicleta é utilizada por dia. Estações otimizadas aumentam esse giro, melhorando o ROI sobre a frota imobilizada.

e. Frente de Governança, Arquitetura e QA

- Tech Lead e Documentação: Responsável pelo Diagrama de Arquitetura, supervisão e integração das frentes de desenvolvimento, elaboração de documentações e pelo mapeamento de requisitos.

7. Funções desempenhadas

Frente	Integrante(s)
Ingestão e Simulação de Eventos	João Henrique Lafetá (jhml@cesar.school); Henrique Roma (hrm@cesar.school)
Engenharia de Dados e Arquitetura	Diego Arruda (dca2@cesar.school); Rodrigo Dubeux (rdmo@cesar.school)
Inteligência Logística e Modelagem	Caio Barreto (cba2@cesar.school)
Economia Empresarial e Visualização	Victor Hora (vht@cesar.school)
Governança, Arquitetura e QA	Ana Beatriz Alves (abxa@cesar.school)

Vale ressaltar que, frente a dependências da finalização da parte inicial, além do planejamento da disciplina somente prever certas atividades e entregas para a AV2, os membros das frentes não implementadas ficaram responsáveis pela consolidação da arquitetura, documentação de processos, estudo de ferramentas e planejamento da execução das próximas etapas.

8. Links úteis

- Repositório GitHub: <https://github.com/anabxalves/citibike-bigdata>
- Fonte de dados original:
https://data.cityofnewyork.us/dataset/Citi-Bike-System-Data/vsnr-94wk/about_data